

LAB: Classify Mushrooms — Edible or Poisonous

8,124 mushrooms, 22 categorical features (cap shape, odor, gill color...). Target: class = 'e' (edible) or 'p' (poisonous). All features are text — we must encode them to numbers first.

Step 1 — Load, Encode, Split

```
# ■ Load and Encode ■
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Download the dataset if it's not present
import os
if not os.path.exists('mushrooms.csv'):
    !wget https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/ag

df = pd.read_csv('mushrooms.csv') # kaggle: 'mushroom classification'
# The downloaded dataset does not have headers, so we need to add them or handle
# For simplicity, let's load it and assume the first column is 'class' for now
# In a real scenario, you would need to define column names.

# For the mushroom dataset from UCI, the 'class' column is the first one.
# The column names are not in the CSV, so we'll need to handle it accordingly.
# Let's assume the dataset from UCI is comma-separated without header.
# The 'class' column is the first column in this dataset.

# Let's reload with proper column names if available, or use integers.
# Based on common usage of this dataset, let's add dummy column names
# and then assign the first column as 'class'.
# There are 23 columns. The first one is the class.
column_names = ['class'] + [f'feature_{i}' for i in range(22)]
df = pd.read_csv('mushrooms.csv', header=None, names=column_names)

print(df.shape) # (8124, 23)
print(df['class'].value_counts()) # p=4208 poisonous, e=3916 edible

# Encode every column: text → integer
# LabelEncoder: 'e'→0, 'p'→1 (and similarly for all features)
le = LabelEncoder()
df_enc = df.copy()
for col in df_enc.columns:
    df_enc[col] = le.fit_transform(df_enc[col])
X = df_enc.drop('class', axis=1) # 22 features
```

```
y = df_enc['class'] # 0=edible, 1=poisonous
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(f'Train: {len(X_train)} rows | Test: {len(X_test)} rows')
```

```
--2026-06-17 10:07:11-- https://archive.ics.uci.edu/ml/machine-learning-databases
Resolving archive.ics.uci.edu (archive.ics.uci.edu)... 128.195.10.252
Connecting to archive.ics.uci.edu (archive.ics.uci.edu)|128.195.10.252|:443... c
HTTP request sent, awaiting response... 200 OK
Length: unspecified
Saving to: 'mushrooms.csv'
```

```
mushrooms.csv      [ <=> ] 364.95K  --.-KB/s    in 0.04s
```

```
2026-06-17 10:07:11 (7.93 MB/s) - 'mushrooms.csv' saved [373704]
```

```
(8124, 23)
class
e    4208
p    3916
Name: count, dtype: int64
Train: 6499 rows | Test: 1625 rows
```

Step 2 — Train a Decision Tree + Visualise

```
# ■ Decision Tree: Train + Visualise ■
# ■ Shallow tree (depth=4) – interpretable ■
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train, y_train)
print(f'Decision Tree (depth=4) accuracy: {dt.score(X_test, y_test):.2%}')
# Text rules – read them like an if-else chain
print(export_text(dt, feature_names=list(X.columns)))
# ■ Find the single most important feature ■
imp = pd.Series(dt.feature_importances_, index=X.columns)
top = imp.idxmax()
print(f'Most important feature: {top} (importance={imp[top]:.3f})')
# Hint: 'odor' almost always comes first!
# odor=None → almost certainly edible. Smell saves lives.
# ■ Visual tree ■
plt.figure(figsize=(18, 8))
plot_tree(dt, feature_names=X.columns, class_names=['Edible', 'Poisonous'],
          filled=True, rounded=True, fontsize=8) # Changed fontsize to an integer
plt.title('Mushroom Decision Tree (max_depth=4)'); plt.tight_layout()
plt.savefig('mushroom_tree.png', dpi=150, bbox_inches='tight')
plt.show()
```

Decision Tree (depth=4) accuracy: 98.22%

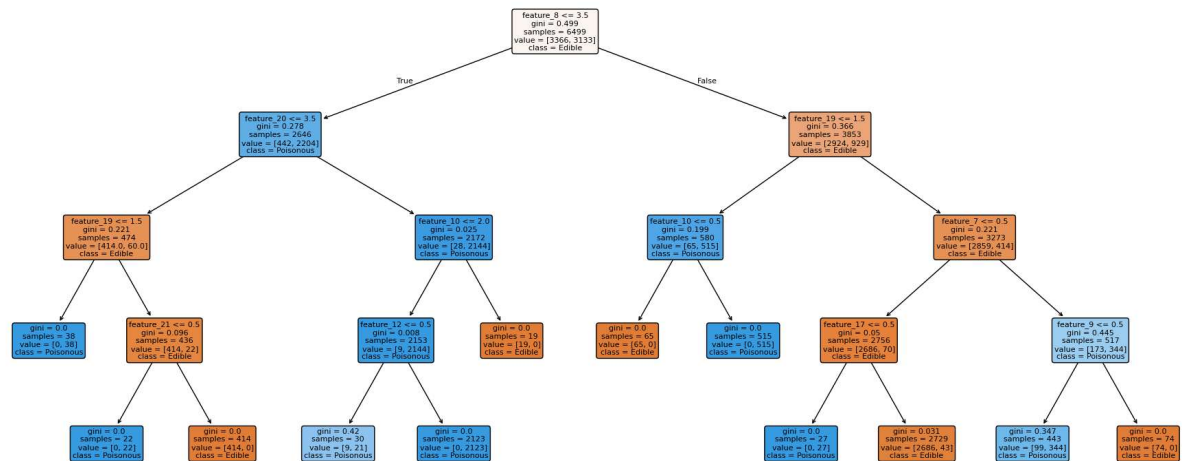
```

|--- feature_8 <= 3.50
|   |--- feature_20 <= 3.50
|   |   |--- feature_19 <= 1.50
|   |   |   |--- class: 1
|   |   |--- feature_19 > 1.50
|   |   |   |--- feature_21 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_21 > 0.50
|   |   |   |   |--- class: 0
|   |--- feature_20 > 3.50
|   |   |--- feature_10 <= 2.00
|   |   |   |--- feature_12 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_12 > 0.50
|   |   |   |   |--- class: 1
|   |   |--- feature_10 > 2.00
|   |   |   |--- class: 0
|--- feature_8 > 3.50
|   |--- feature_19 <= 1.50
|   |   |--- feature_10 <= 0.50
|   |   |   |--- class: 0
|   |   |--- feature_10 > 0.50
|   |   |   |--- class: 1
|   |--- feature_19 > 1.50
|   |   |--- feature_7 <= 0.50
|   |   |   |--- feature_17 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_17 > 0.50
|   |   |   |   |--- class: 0
|   |   |--- feature_7 > 0.50
|   |   |   |--- feature_9 <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- feature_9 > 0.50
|   |   |   |   |--- class: 0

```

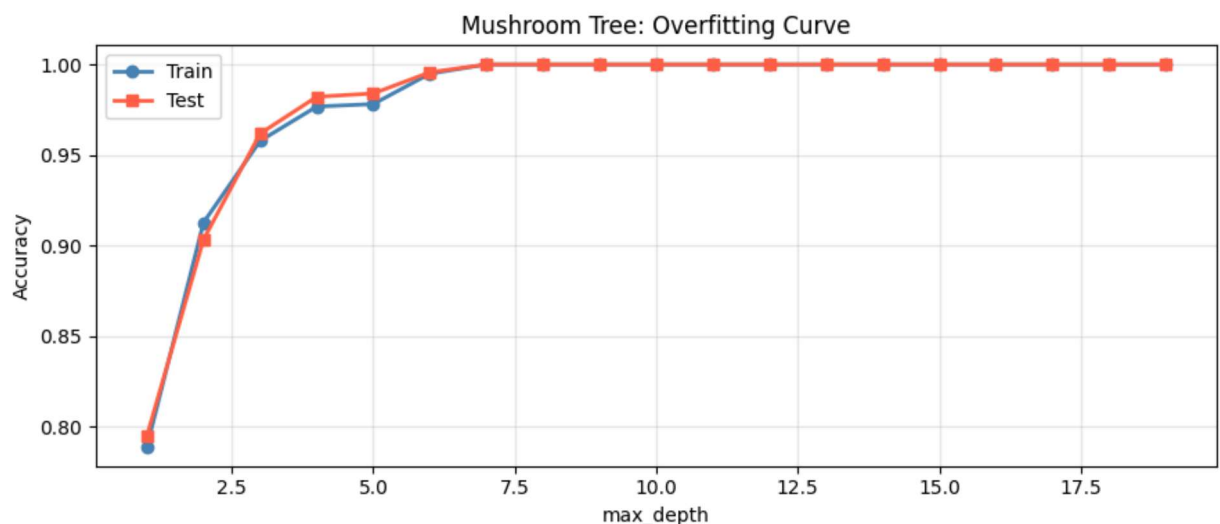
Most important feature: feature_8 (importance=0.367)

Mushroom Decision Tree (max_depth=4)



Step 3 — Depth Sweep: Find the Overfit Point

```
# ■ Depth Sweep ■
train_acc, test_acc = [], []
for d in range(1, 20):
    tree = DecisionTreeClassifier(max_depth=d, random_state=42)
    tree.fit(X_train, y_train)
    train_acc.append(tree.score(X_train, y_train))
    test_acc.append(tree.score(X_test, y_test))
plt.figure(figsize=(9, 4))
plt.plot(range(1,20), train_acc, 'o-', lw=2, label='Train', color='steelblue')
plt.plot(range(1,20), test_acc, 's-', lw=2, label='Test', color='tomato')
plt.xlabel('max_depth'); plt.ylabel('Accuracy')
plt.title('Mushroom Tree: Overfitting Curve')
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
best = range(1,20)[test_acc.index(max(test_acc))]
print(f'Best depth: {best}, Test accuracy: {max(test_acc):.2%}')
```

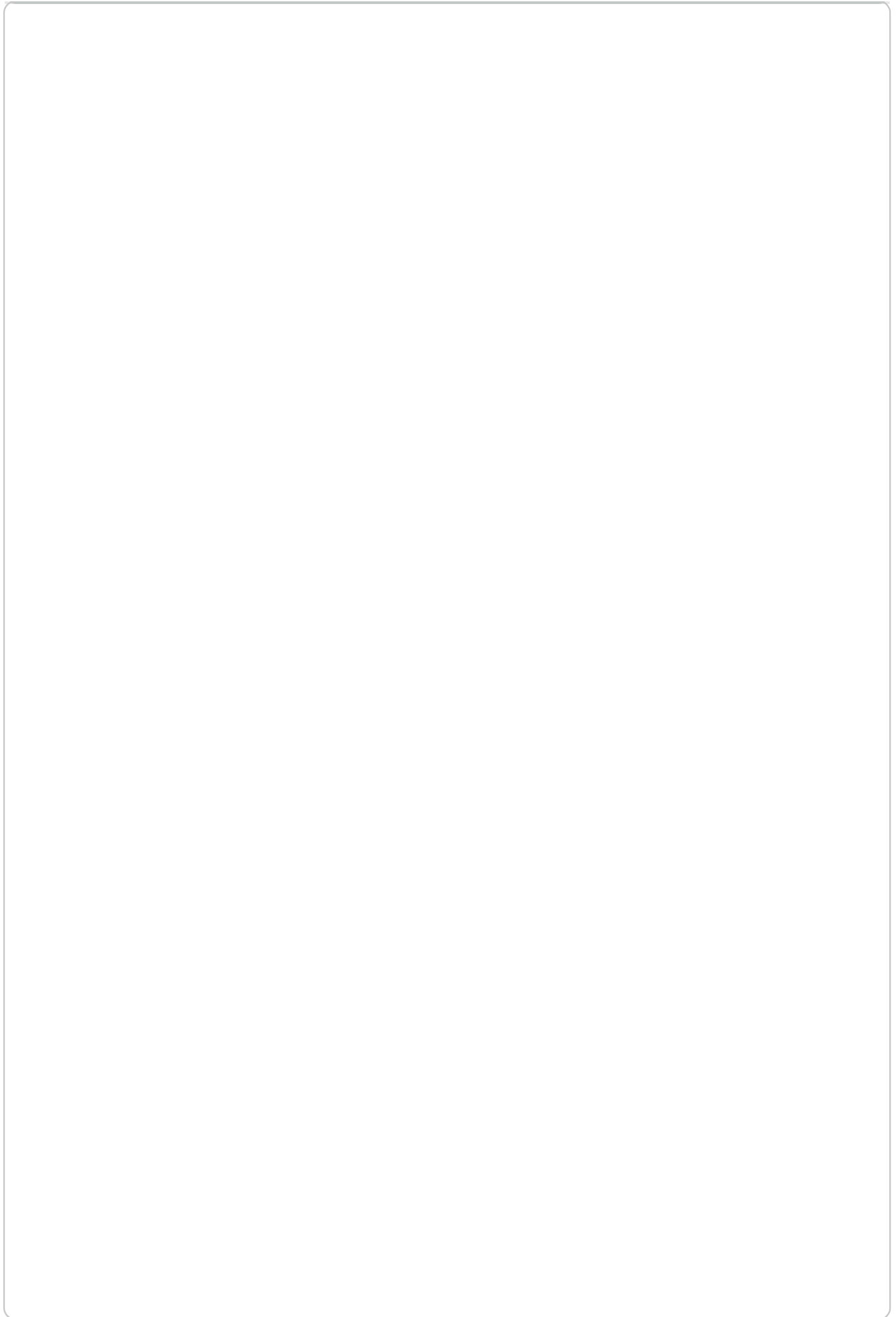


Best depth: 7, Test accuracy: 100.00%

Step 4 — Random Forest: Boost Accuracy + Full Importance Chart

```
# ■ Random Forest: Full Evaluation + Ablation Test ■
rf = RandomForestClassifier(n_estimators=100, oob_score=True, random_state=42)
rf.fit(X_train, y_train)
print(f'Random Forest Test Accuracy: {rf.score(X_test, y_test):.2%}')
print(f'Random Forest OOB Accuracy: {rf.oob_score_: .2%}')
print(classification_report(y_test, rf.predict(X_test), target_names=['I', 'V']))
# Cross-validation for robustness check (5 different splits)
cv_scores = cross_val_score(rf, X, y, cv=5, scoring='accuracy')
print(f'5-Fold CV: {cv_scores.mean():.2%} ± {cv_scores.std():.2%}')
# Feature importance bar chart
rf_imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
rf_imp.plot(kind='barh', figsize=(9, 8), color='mediumseagreen', edgecolor='black')
plt.title('Random Forest Feature Importance – Mushroom Dataset', fontsize=12)
plt.xlabel('Importance Score'); plt.tight_layout(); plt.show()
# EXPERIMENT: Remove 'odor' and retrain – does accuracy drop dramatically?
X_no_odor = X.drop(columns=['feature_8']) # Changed 'odor' to 'feature_8'
X_train2, X_test2, y_train, y_test = train_test_split(X_no_odor, y, test_size=0.2, random_state=42)
```

```
rf2 = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train, y_train)
print(f'\nAccuracy WITHOUT odor feature: {rf2.score(X_test, y_test):.2%}')
print(f'Accuracy WITH odor feature: {rf.score(X_test, y_test):.2%}')
```



Random Forest Test Accuracy: 100.00%
 Step 5 - Predict for New Mushrooms

precision recall f1-score support

the test set to predict

index]]

le_index]

e)[0]

else 'POISONOUS'

el == 0 else 'POISONOUS'

,